

LABORATÓRIO DE BANCO DE DADOS · 2026/2

Introdução ao Docker

Containers, imagens, volumes, redes e Compose

Atividade EaD 1 · Turmas P01 e P02

Prof. Dr. Márcio Silva

Faculdade de Computação

Universidade Federal de Mato Grosso do Sul



Sobre este material

Este deck é uma **adaptação** do curso *Introdução ao Docker* produzido pelo **Insight Data Science Lab** — **UFC**.

Autores do material de origem

Prof. Gustavo Coutinho · Prof. Regis Pires · Lucas Peres
insightlab.ufc.br

O que mudou nesta versão

Identidade visual da disciplina, comandos revisados para o Docker atual e notas de atualização marcadas com o selo

ATUALIZADO

O curso de origem, por sua vez, referencia o material de Docker e Docker Compose da Alura.

Agenda

1 · Primeiros passos

Das máquinas virtuais aos containers.
O que o Docker é — e o que ele não é.

2 · O que são imagens?

Criar e executar containers. O sistema de camadas.

3 · Volumes

Persistir dados além do ciclo de vida do container.

4 · Construir imagens

Dockerfile, build, publicação no Docker Hub.

5 · Comunicação

Redes do Docker e containers que conversam entre si.

6 · Docker Compose

Vários containers descritos em um arquivo YAML.

PARTE 01

Primeiros passos com Docker

Como surgiram os containers — e por que eles substituíram as máquinas virtuais em boa parte dos casos.



Como hospedávamos aplicações antes?

Para vários serviços, usávamos **várias máquinas físicas**.

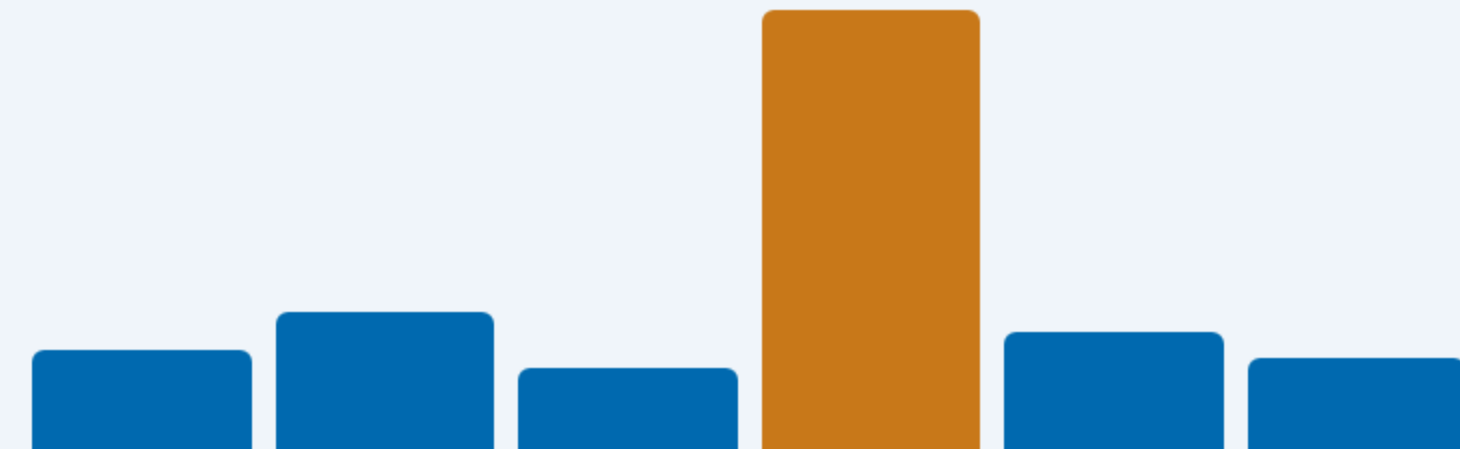
Aplicação 1	Aplicação 2	Aplicação n
S.O.	S.O.	S.O.
Máquina física	Máquina física	Máquina física

Capacidade subutilizada

Máquinas ociosas na maior parte do tempo, desperdiçando poder computacional.

- O uso da capacidade máxima é **sazonal**
- Paga-se pelo pico, opera-se na média

Processamento na máquina real



Um pico raro dimensiona a máquina inteira.

Solução: virtualizar os serviços



As máquinas virtuais **otimizaram o uso da capacidade de processamento** disponível: várias aplicações passam a dividir o mesmo hardware.

Quais são as desvantagens das MV?

Cada máquina virtual carrega um **sistema operacional inteiro** — e o custo que vem com ele.

Configuração

Um SO a instalar e ajustar por máquina.

Atualização

Patches e upgrades multiplicados por n.

Segurança

Cada SO é uma superfície de ataque própria.

Recursos

Armazenamento, processamento e memória reservados antes de qualquer aplicação rodar.

Resolvemos o problema das máquinas físicas com as virtuais. **E agora, como resolver o problema das máquinas virtuais?**

Caracterizando um container

Um container é um **ambiente de virtualização** — mas sem carregar um sistema operacional inteiro junto.

Menos processamento

Não há um SO convidado consumindo recursos em paralelo.

Menos manutenção

Nada de configurar e atualizar um SO por serviço.

Início quase imediato

Criação e finalização em segundos, não em minutos.

Por que não rodar as aplicações direto no SO?

1 · Concorrência entre portas

Duas aplicações que querem a mesma porta não coexistem.

2 · Consumo unilateral da CPU

Um processo mal comportado degrada todos os outros.

3 · Conflito de dependências

Versões diferentes da mesma biblioteca disputam o mesmo sistema.

Com containers, ganhamos

Controle sobre o uso de cada recurso

Velocidade para criar e destruir serviços

Isolamento de dependências e bibliotecas

Menor consumo de processamento que as MV

Docker

Alternativa de virtualização em que o **kernel da máquina hospedeira é compartilhado** com o software em operação.



Compare com a pilha das MV: sumiu uma camada inteira de sistemas operacionais convidados.

Hypervisor × virtualização no nível do SO

Hypervisor

Apresenta aos sistemas convidados uma **plataforma operacional virtual** e gerencia a execução deles.

- Várias instâncias de vários SOs compartilham o hardware virtualizado
- Linux, Windows e macOS podem rodar na mesma máquina x86 física

×

Virtualização no nível do SO

Todas as instâncias — os containers — **compartilham um único kernel**.

- O espaço de usuário pode diferir: distribuições Linux distintas sobre o mesmo kernel
- Não é possível rodar um kernel diferente do host

Docker *não* é um hypervisor

As aplicações **não rodam "em cima"** do Docker Engine, como rodariam sobre um hypervisor.

Fora do caminho

O Docker Daemon **não fica no caminho de execução** da aplicação.

Paredes, não andares

Os containers têm "paredes" entre si — isolamento lateral, não uma camada acima.

Só mais um processo

O Docker Daemon é **apenas mais um processo** do sistema operacional.

Fonte: blog.mikesir87.io/2017/05/docker-is-not-a-hypervisor

Docker — as tecnologias

Docker Compose

Ferramenta para **definir e executar** aplicações formadas por vários containers.

Docker Hub

Repositório com **centenas de milhares de imagens** prontas para uso.

Docker Swarm

Orquestração nativa: containers distribuídos em um **cluster**, com controle de quantidade, registro, deploy e update de serviços.

Nota de contexto **ATUALIZADO**

Na prática de mercado, a orquestração hoje é dominada pelo **Kubernetes**. O Swarm continua existindo e é mais simples de operar — bom para entender o conceito de cluster.

Onde praticar

Play with Docker

Ambiente online, sem instalar nada. Acesse, clique em **+ ADD NEW INSTANCE** e use como um terminal comum.

labs.play-with-docker.com

Docker na sua máquina

Docker Engine no Linux, Docker Desktop no Windows e macOS. É o que vamos usar no resto da disciplina.

docs.docker.com/get-docker

Instalado o ambiente, confirme a versão antes de seguir.

```
docker version
```

O *Hello World* dos containers

```
docker run hello-world
```

Baixa a imagem, cria um container a partir dela, executa e encerra.

```
$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Glossário (1 de 3)

Imagem

Pacote com todas as dependências e informações necessárias para criar um container.

Dockerfile

Arquivo de texto com as instruções de como construir uma imagem.

Build

A ação de criar uma imagem a partir de um Dockerfile.

Container

Instância em execução de uma imagem.

Volume

Sistema de arquivos gravável usado pelo container. Fica no host e é gerenciado pelo Docker — sobrevive à remoção do container.

Glossário (2 de 3)

Tag

Rótulo aplicado a uma imagem, para distinguir versões da mesma imagem.

Registro

Serviço que dá acesso aos repositórios. O padrão para imagens públicas é o Docker Hub; empresas costumam manter registros privados.

Repositório

Coleção de imagens relacionadas, identificadas por tags de versão.

Docker Hub

Registro público para publicar e obter imagens. Hospeda repositórios públicos e privados e integra-se com GitHub e Bitbucket.

Glossário (3 de 3)

Compose

Ferramenta de linha de comando e formato YAML para definir e executar aplicações de vários containers.

Cluster

Coleção de hosts Docker exposta como um único host virtual. A aplicação pode escalar para várias instâncias distribuídas entre eles.

Orquestrador

Ferramenta que simplifica a gestão de clusters e hosts: executa, distribui, dimensiona e repara cargas de trabalho em um conjunto de nós — por CLI ou interface gráfica. Exemplos: Kubernetes, Docker Swarm.

PARTE 02

O que são imagens?

Criar e executar containers. Por que uma imagem chega em pedaços — e o que isso economiza.



O que aconteceu no `docker run`?



A imagem é, no fundo, **uma série de instruções** que o Docker segue para montar o container. O comando `docker run` é o atalho que faz os quatro passos de uma vez.

A imagem chega em pedaços

```
docker run alpine
```

Repare no download: a imagem vem dividida em várias camadas, baixadas em paralelo.

Pergunta

Por que **nada aconteceu** quando o download terminou?

Guarde a pergunta — a resposta aparece em três slides.

Algumas imagens e seus tamanhos

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
busybox	latest	19485c79a9bb	6 weeks ago	1.22MB
alpine	latest	965ea09ff2eb	16 hours ago	5.55MB
ubuntu	latest	cf0f3ca922e0	3 days ago	64.2MB
debian	stable-slim	eece114ab04d	5 days ago	69.2MB
debian	latest	8e9f8546050d	5 days ago	114MB
centos	latest	0f3e07c0138f	2 weeks ago	220MB

Compare com uma máquina virtual: um SO convidado raramente sai por menos de 1 GB.

A escolha da imagem base tem consequência direta no tamanho do que você distribui. **Alpine** é a base minimalista mais usada por isso.

Quais containers estão rodando?

`docker ps`

Só os containers **em execução**.

`docker ps -a`

Todos — inclusive os **parados**.

```
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              NAMES
3hg567caz645   alpine        "sleep 20"              2 minutes ago Exited (0) 1 minute ago   nifty_bohr
7bgy35cfd4sg   hello-world   "/hello"                5 minutes ago Exited (0) 5 minutes ago   epic_curie
```

Quando um container termina de executar, ele não some: fica **parado** (**Exited**), ocupando espaço até ser removido.

Por que nada aconteceu?

Porque o container precisa de **algo para fazer**. Sem um processo em execução, ele nasce e morre.

```
docker run alpine echo "Ola Mundo"
```

O comando vai depois do nome da imagem. O container vive enquanto esse processo viver.

Regra que explica quase tudo

Um container é **um processo**. Terminou o processo, terminou o container.

Escolhendo a versão: tags

```
docker run ubuntu:18.04 cat /etc/issue
```

A tag vem depois dos dois-pontos. Sem tag, o Docker assume `latest`.

Onde ver as tags disponíveis

Na página da imagem no Docker Hub, aba *Tags*.

hub.docker.com

`latest` não significa "a mais recente" **ATUALIZADO**

É só a tag padrão, e pode apontar para qualquer versão. Em produção, **fixe a tag** — ou o build de amanhã não será o de hoje.

Executando em segundo plano

```
docker run -d alpine sleep 20
```

A flag `-d` (*detached*) devolve o terminal e deixa o container rodando ao fundo.

Por que o `sleep` é necessário aqui?

Sem um processo que dure, o container encerra imediatamente e você nunca o veria em `docker ps`. O `sleep 20` dá a ele 20 segundos de vida.

```
docker stop 3hg567caz645
```

Para um container em execução.

```
docker run --rm alpine echo "Olá"
```

Remove o container assim que ele termina.

O `run` sempre cria um container novo

Esta é a confusão mais comum de quem está começando. `run` não "abre" um container existente — ele **fabrica outro**, do zero, a partir da imagem.

`docker run`

Cria um container novo e o executa.
Rodou dez vezes? Dez containers.

`docker start`

Religa um container que já existe,
preservando o que havia nele.

`docker exec`

Executa um comando adicional **dentro**
de um container que já está rodando.

```
docker exec 3hg56 ls
```

Entrando no container

```
docker run -it alpine
```

`-i` mantém a entrada padrão aberta · `-t` aloca um terminal. Juntas, ligam o seu terminal ao do container.

Para religar um container parado

```
docker start -a -i <id>
```

- `-a` (attach) anexa os terminais
- `-i` (interactive) permite interagir

As ferramentas do host não são as do container

Compare a mesma pergunta feita nos dois lados:

```
bash --version
```

```
docker run alpine bash --version
```

No Alpine o segundo falha — a imagem nem traz o bash.

Os dois estados de um container

Running

O processo principal está vivo

docker stop



docker start

Stopped

O container existe, mas nada executa

```
docker rm <id>
```

Remove um container.

```
docker container prune
```

Remove **todos** os containers parados de uma vez.

Removendo imagens

```
docker images
```

Lista as imagens presentes na máquina.

```
docker rmi <imagem>
```

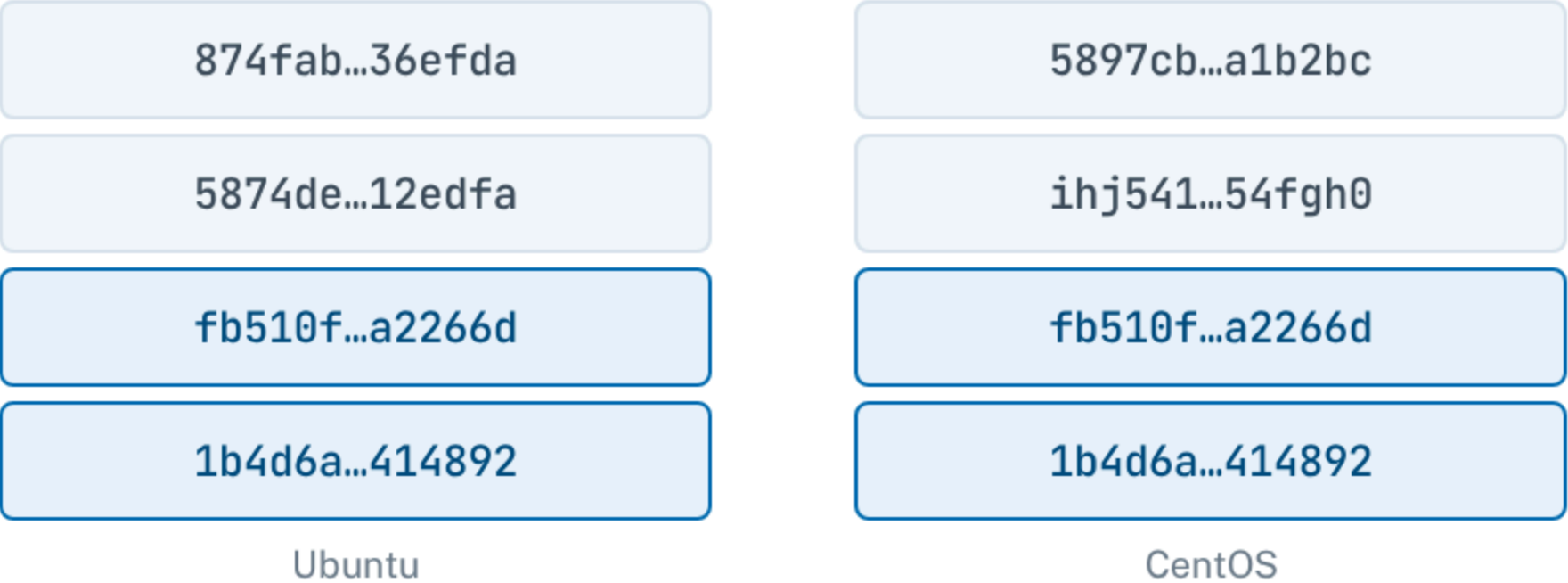
Remove uma imagem.

Atenção à ordem

Uma imagem **só é removida se não houver mais containers criados a partir dela** — nem mesmo os parados. Limpe os containers primeiro.

Layered File System

Toda imagem é composta por **uma ou mais camadas** — foi o que vimos no download, chegando em pedaços.



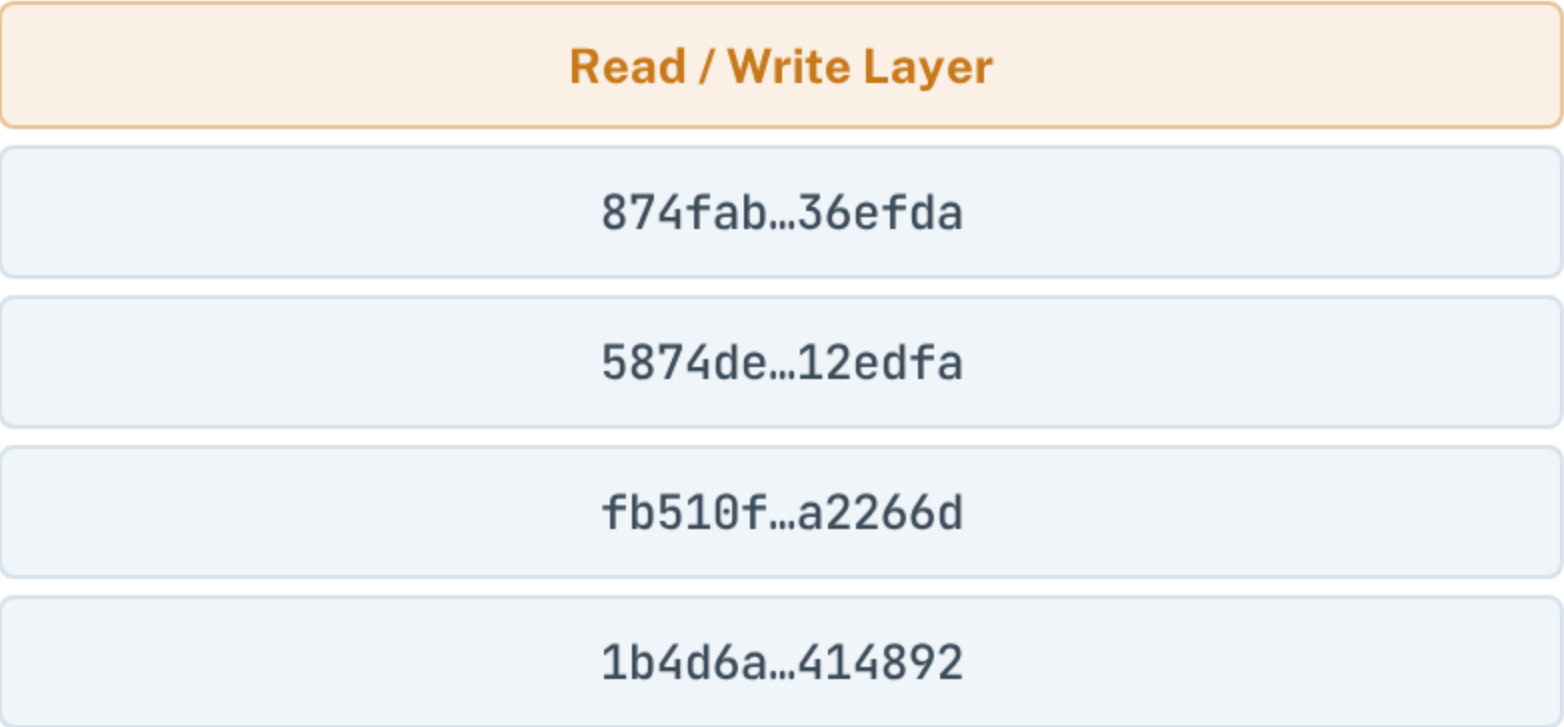
As camadas em azul são **as mesmas** nas duas imagens: baixadas e armazenadas uma única vez.

As camadas da imagem são somente leitura

Mas então como conseguimos criar arquivos dentro do container?

Não escrevemos na imagem. Ao criar o container, o Docker adiciona **uma nova camada gravável por cima** — e é só nela que as alterações acontecem.

A imagem permanece intacta. É isso que permite dez containers partirem da mesma imagem sem interferir uns nos outros.



Somente as camadas de baixo são read-only

Vários containers, uma única imagem



Três containers, três camadas graváveis — e **uma só cópia** das camadas da imagem no disco.

PARTE 03

Volumes e armazenamento

A camada gravável morre com o container. Onde guardar o que precisa sobreviver — como os dados de um banco.



Removeu o container, perdeu os dados

Ao remover um container, removemos junto a **camada de leitura e escrita** — e tudo que foi gravado nela.

Por que isso importa nesta disciplina

Se o banco de dados roda em um container e os arquivos ficam na camada gravável, **um docker rm apaga o banco inteiro**. É exatamente o que os volumes evitam.

O lugar próprio para essa persistência são os **volumes de dados**.

O que é um volume

Criar um volume é **apontar uma pasta do container para uma pasta no Docker Host**. O container pode ser removido; a pasta no host fica intacta.

```
docker run -d -p 8080:80 -v "/usr/share/nginx/html" nginx
```

A flag `-v` declara o volume. Aqui, sem indicar a pasta do host — o Docker escolhe uma.

Mas qual pasta ele escolheu?

Inspecione o container e procure por `Mounts`:

```
docker inspect 316
```

```
docker inspect 316 | grep -i -A 5 mounts
```

Também vale para portas

`-p 8080:80` liga a porta **8080 do host** à porta **80 do container**. A ordem é sempre host:container — a mesma do

`-v`.

Escolhendo a pasta do host

Quatro formas equivalentes de dizer a mesma coisa:

```
docker run -p 8080:80 -v "/root/www:/usr/share/nginx/html" nginx  
docker run -p 8080:80 -v "$HOME/www:/usr/share/nginx/html" nginx  
docker run -p 8080:80 -v ~/www:/usr/share/nginx/html nginx  
docker run -p 8080:80 -v "~/www:/usr/share/nginx/html" nginx # cuidado
```

A última linha é uma armadilha

Entre aspas, o `~` **não é expandido** pelo shell: o Docker cria uma pasta chamada literalmente `~` no diretório atual. Prefira `$HOME` quando precisar de aspas.

Teste rápido: crie um arquivo dentro do container e abra-o na sua máquina — e vice-versa.

Rodando código dentro de um container

Se a pasta é compartilhada nos dois sentidos, você pode **escrever o código na sua máquina e compilá-lo e executá-lo dentro do container**.

Sem instalar nada

Use uma linguagem ou versão que não está instalada no seu sistema.

Ambiente descartável

Quebrou? Apague o container e comece de novo em segundos.

Igual para todos

O ambiente de quem escreveu é o mesmo de quem corrige. Fim do "na minha máquina funciona".

Ou seja: **seu ambiente de desenvolvimento pode viver dentro de um container** — inclusive o MySQL e o PostgreSQL que usaremos na disciplina.

PARTE 04

Construindo nossas próprias imagens

Até aqui usamos imagens prontas. Agora escrevemos a receita da nossa.



O Dockerfile

Para criar uma imagem, escrevemos a **receita** dela: um arquivo de texto com as instruções que o Docker vai executar, em ordem.

Nome do arquivo

Por convenção, `Dockerfile`, sem extensão. Pode-se usar um nome descritivo com a extensão `.dockerfile` — por exemplo `mongo.dockerfile` — quando há vários no mesmo projeto.

Toda imagem parte de outra

Salvo raras exceções, construímos **sobre uma imagem base**: um Alpine, um Debian, um Python já pronto. A primeira instrução declara qual.

Montando o Dockerfile, linha a linha

```
FROM alpine:3.9
```

A imagem base e sua versão.

```
FROM alpine:3.9  
LABEL maintainer="Insight Data Science Lab"
```

O mantenedor da imagem. **ATUALIZADO** O material original usa `MAINTAINER`, descontinuado desde o Docker 1.13.

```
FROM alpine:3.9  
LABEL maintainer="Insight Data Science Lab"  
RUN apk add --no-cache mongodb
```

RUN executa um comando **durante o build**, e o resultado vira uma camada.

Cada instrução é uma camada

Por isso a ordem importa: instruções que mudam pouco vão em cima, para reaproveitar cache; o que muda a cada build vai por último.

O Dockerfile completo

```
FROM alpine:3.9
LABEL maintainer="Insight Data Science Lab"
RUN apk add --no-cache mongodb
VOLUME /data/db
EXPOSE 27017
CMD [ "mongod", "--bind_ip", "0.0.0.0" ]
```

- **VOLUME** — declara o ponto de montagem dos dados
- **EXPOSE** — documenta a porta que o serviço usa
- **CMD** — o comando que roda quando o container inicia

EXPOSE não publica a porta sozinho: quem faz isso é o **-p** no **run**.

Construindo a imagem

```
docker build -f Dockerfile -t insightlab/mongo:1.0 .
```

Com o arquivo indicado explicitamente.

```
docker build -t insightlab/mongo:1.0 .
```

Equivalente, quando o arquivo se chama `Dockerfile` e está na pasta atual.

-f

Qual arquivo usar como receita.

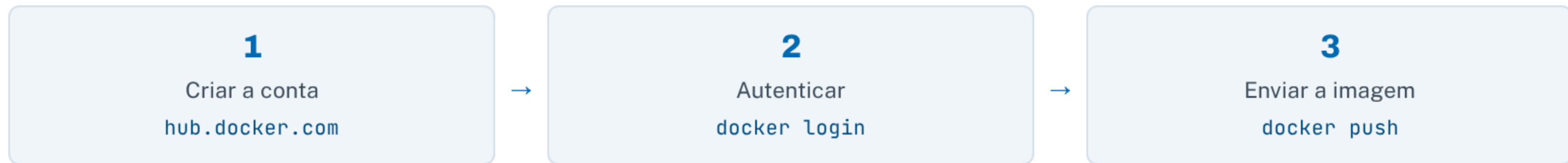
-t

Nome e tag da imagem resultante.

O ponto final

Não é enfeite: é o **contexto do build**, a pasta cujo conteúdo será enviado ao Docker.

Publicando no Docker Hub



```
docker push insightlab/mongo:1.0
```

Envia a imagem para o registro.

```
docker pull insightlab/mongo:1.0
```

Baixa a imagem em qualquer outra máquina.

O nome da imagem precisa começar pelo seu usuário no Hub — é isso que define para onde o push vai.

Executando a imagem que construímos

Servidor MongoDB

```
docker run -d -p 27017:27017 -v $HOME/db:/data/db insightlab/mongo:1.0
```

Porta publicada, dados persistidos em \$HOME/db.

Cliente mongo-express

```
docker run -p 8081:8081 -e ME_CONFIG_MONGODB_SERVER="172.17.0.2" mongo-express
```

-e define uma variável de ambiente dentro do container. O IP fixo aqui é frágil — o próximo módulo resolve isso.

ENTRYPOINT × CMD

ENTRYPOINT

Especifica **o comando** que roda quando o container inicia.

- Ponto de entrada padrão: `/bin/sh -c`

×

CMD

Especifica **os argumentos** passados ao ENTRYPOINT.

- O Docker não tem CMD padrão

Por isso, ao executar:

```
docker run -it alpine bash
```

o que roda de fato é `/bin/sh -c bash`.

ENTRYPOINT e CMD na prática

Dockerfile

```
FROM alpine
ENTRYPOINT ["/bin/ping"]
CMD ["localhost"]
```

Build

```
docker build -t alpine-ping .
```

Usando

```
docker run --rm alpine-ping
```

Sem argumento: usa o CMD padrão e faz ping em localhost.

```
docker run --rm alpine-ping 8.8.8.8
```

Com argumento: o **CMD** é substituído, o ENTRYPOINT permanece.

Regra prática: **ENTRYPOINT** define o que a imagem é; **CMD** define o padrão que o usuário pode trocar.

PARTE 05

Comunicação entre containers

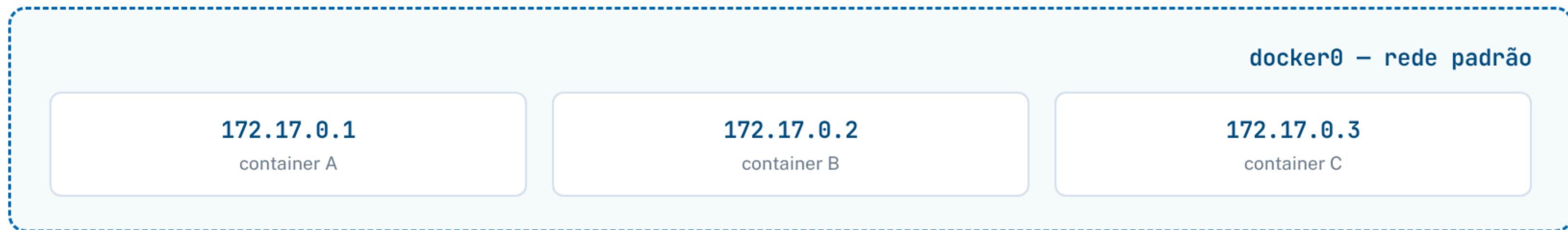
Uma aplicação real tem partes. Cada parte em seu container — e todas precisando conversar.



A rede padrão do Docker

```
docker network ls
```

Lista as redes existentes. O Docker já cria uma *default network* por conta própria.



Dentro dessa rede, os containers **se comunicam livremente** por IP. Para ver o endereço de um deles, use `docker inspect <id>` ou, de dentro, `hostname -i`.

O problema do endereço que muda

Cada container instanciado recebe **um IP novo**. E a aplicação precisa saber onde está o banco de dados.

Por que dói

Escrever 172.17.0.2 na configuração da aplicação funciona hoje e quebra amanhã, no primeiro `docker run`.

`--name` não resolve na rede padrão

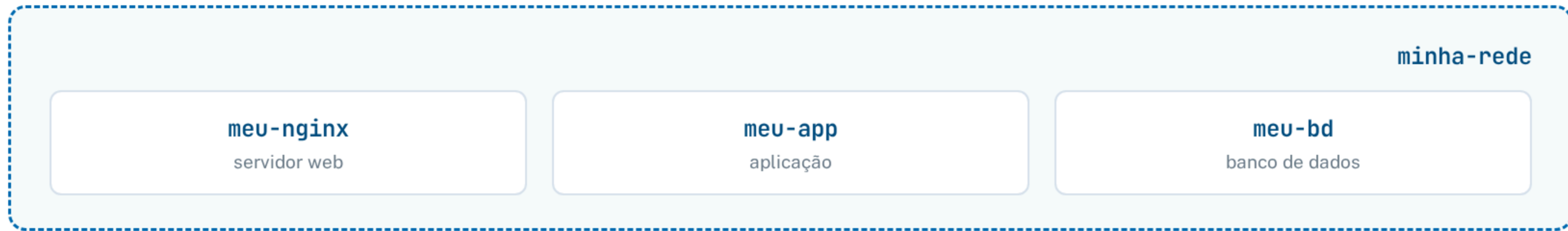
Dá para nomear o container, mas a **rede padrão não registra hostnames**: o nome não é resolvível pelos outros containers.

A saída

Criar **uma rede própria**. Nela, os containers se enxergam **pelo nome**, e o IP deixa de importar.

Criando uma rede própria

```
docker network create --driver bridge minha-rede
```



```
docker run -it --name meu-alpine --network minha-rede alpine
```

A partir daqui, um container alcança o outro escrevendo `ping meu-bd` — sem nunca saber o IP.

Ligando MongoDB e mongo-express

Servidor

```
docker run -d -v $HOME/db:/data/db --name=mongo --network=minha-rede insightlab/mongo:1.0
```

Cliente

```
docker run -p 8080:8081 -e ME_CONFIG_MONGODB_SERVER="mongo" --network=minha-rede mongo-express
```

Compare com o slide anterior à rede

Onde havia "172.17.0.2", agora há "mongo" — o nome do container. Essa é a diferença que a rede própria compra.

PARTE 06

Docker Compose

Toda a linha de comando dos módulos anteriores, escrita uma vez em um arquivo e versionada junto com o projeto.



Por que não instanciar tudo na mão

Instanciar um container manualmente já é arriscado — esquecer uma flag, errar uma porta. Com **três, quatro, dez containers**, o problema se multiplica.

Na linha de comando

O conhecimento de como subir o sistema mora na memória de quem digitou. Ninguém mais consegue reproduzir.

No Docker Compose

Um arquivo **YAML** descreve tudo o que deve acontecer. Vai para o repositório junto com o código.

Aplicações reais, em geral, têm bem mais que dois ou três containers.

O que o Compose entrega

Ambientes isolados

Vários ambientes independentes convivendo em um único host.

Dados preservados

O conteúdo dos volumes sobrevive à recriação dos containers.

Recriação seletiva

Só os serviços que mudaram são reconstruídos.

Portabilidade

Variáveis permitem mover a mesma composição entre ambientes.

E, no ciclo de vida da aplicação

Iniciar, parar e reconstruir serviços

Ver o status do que está rodando

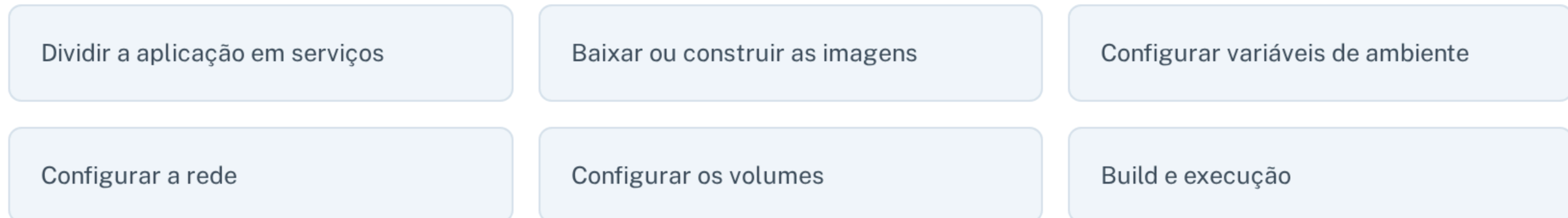
Acompanhar os logs em fluxo contínuo

Executar um comando avulso em um serviço

O fluxo de trabalho



Tarefas usuais



Comandos do Compose

```
docker compose up -d      # sobe tudo em segundo plano
docker compose down       # derruba e remove containers e rede
docker compose start      # inicia serviços já criados
docker compose stop       # para sem remover
docker compose build       # reconstrói as imagens
docker compose logs       # mostra os logs dos serviços
docker compose events     # acompanha eventos em tempo real
docker compose exec <serviço> <comando>
```

Note o espaço **ATUALIZADO**

O material de origem usa `docker-compose` com hífen (Compose V1, em Python, descontinuado). O comando atual é `docker compose`, subcomando do próprio Docker. Os dois ainda funcionam em muitas instalações — prefira o novo.

Exemplo de docker-compose.yml

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - ./code
      - logvolume01:/var/log
    depends_on:
      - redis
  redis:
    image: redis
```

- **services** — cada chave é um container
- **build: .** — constrói a partir do Dockerfile local
- **image:** — usa uma imagem pronta do registro
- **ports / volumes** — os mesmos `-p` e `-v` da linha de comando

Duas mudanças ATUALIZADO

O original traz `version: '3'` na primeira linha e `links:` para ligar os serviços. Ambos são **legados**: a versão é ignorada pelo Compose atual e `links` foi substituído por `depends_on` — os serviços já se enxergam pelo nome, na rede que o Compose cria sozinho.

O mesmo exemplo do MongoDB, agora em YAML

```
services:
  mongo:
    image: mongo
    volumes:
      - ./db:/data/db
  mongo-express:
    image: mongo-express
    ports:
      - "8080:8081"
    environment:
      ME_CONFIG_MONGODB_SERVER: mongo
    depends_on:
      - mongo
```

Compare com os dois `docker run` da Parte 5.

- Nenhuma rede precisou ser criada à mão — o Compose cria uma para o projeto
- O endereço do banco é **o nome do serviço**: mongo
- Um único `docker compose up` sobe os dois

O que você precisa levar daqui

Container é processo

Nasce com um comando, morre com ele. Não é uma máquina virtual pequena.

Imagem é receita em camadas

Somente leitura, compartilhadas entre imagens. O container só acrescenta a camada gravável.

Volume é o que sobrevive

Sem volume, remover o container apaga os dados — inclusive os do banco.

Rede própria dá nomes

Containers se acham pelo nome do serviço; IP fixo em configuração é armadilha.

Dockerfile é versionável

O ambiente deixa de ser conhecimento tácito e vira arquivo no repositório.

Compose orquestra o conjunto

Um YAML no lugar de uma sequência de comandos que ninguém lembra.

Para continuar

Questionário no AVA

Responder ao questionário desta atividade **registra a sua frequência** do dia. Não responder conta falta.

Documentação oficial

docs.docker.com

A referência que envelhece menos.

Desafio prático

Repositório de exercícios do material de origem:
github.com/InsightLab/docker-introduction-challenge

Nas próximas atividades EaD

MySQL e PostgreSQL — **ambos rodando em containers**. O que foi visto aqui é pré-requisito.

FIM DA ATIVIDADE

Dúvidas?

Traga-as para o fórum da disciplina no AVA ou para o próximo encontro presencial.

Material de origem: curso *Introdução ao Docker* — Insight Data Science Lab, UFC.
Prof. Gustavo Coutinho · Prof. Regis Pires · Lucas Peres.

